```
4.  print(gcd(987654321987654321,
            12345678912345678912345678))
5.  print(factor(x^4 - 1))
6.  print(GF(7)['x'].random_element( ))
7.  print(gcd(x^3 - 1, x^2 - 1))
8.  print(power_mod(5, 3, 5))
9.  print(power_mod(5, 3, 50))
```

```
: 733 * 4637 * 329401 * 974293 * 1360682471 * 106007173861643 * 7061709990156159479
True
False
9
(x^2 + 1)*(x + 1)*(x - 1)
x + 5
x - 1
0
25
```

Figure 1: A few NTL one-liners

Now, let us try to understand the one-liners. Line 1 calls the function *factor( )* on a very large integer and prints its prime factorisation (Sage uses fast number-theory libraries to accomplish this). Line 2 uses the function *is_prime( )* to test whether the integer 7061709990156159479 is prime and prints *True* or *False* accordingly. Line 3 performs the same primality test for the number 706170999015615947 8. Line 4 computes and prints the greatest common divisor of the two large integers 987654321987654321 and 12345678912345678912345678 9. Line 5 factors the polynomial x^4-1 over the integers and prints the resulting factorisation. Line 6 constructs a polynomial ring over the finite field GF(7) and prints a randomly generated polynomial from GF(7)['x']. I am not going to cover this concept in detail here, but it is important to remember that finite fields play a crucial role in cryptography, which is a major component of cybersecurity. Line 7 computes and prints the greatest common divisor of the two polynomials x^3-1 and x^2-1. Line 8 computes 5^3 mod 5 using the function *power_mod( )* and prints the result. Finally, Line 9 computes 5^3 mod 50. All these operations are handled by SageMath's high-performance arithmetic and algebra backend libraries like NTL, FLINT, etc, enabling efficient computation even with very large integers and polynomials.

Since the calculations involved in the above program are computationally intense, in this article also we are using CoCalc to execute the SageMath programs. Upon executing the program 'ntl40.sage', the output shown in Figure 1 is obtained. The first line printed in this figure shows that the factors of the number 111111111111111111111111111111111111111111111111 11111111111111 are 733, 4637, 329401, 974293, 1360682471, 106007173861643, and 7061709990156159479. The second line confirms that the number 7061709990156159479 is prime. This is obvious, as this number is one of the prime factors obtained while executing Line 1 of the program 'ntl40.sage'. Line 3 tells us that the number 7061709990156159478 is not a prime and is hence composite. Line 4 tells us that the GCD of the numbers 987654321987654321 and 12345678912345678912345678 9 is 9. I urge you to take a minute to verify this fact. Line 5 in Figure 1 tells us that the factors of the polynomial $(x^4 - 1)$ are $(x^2 + 1)$, $(x + 1)$, and $(x - 1)$. Line 6 shows the randomly generated polynomial from GF(7)['x'], which in this case is $(x + 5)$. Line 7 shows that the GCD of the two polynomials $(x^3 - 1)$ and $(x^2 - 1)$ is $(x - 1)$. Line 8 shows that 5^3 mod 5 = 125 mod 5 = 0. Finally, Line 9 shows that 5^3 mod 50 = 125 mod 50 = 25.

Implementing a digital signature

Now, it is time to implement a digital signature. As mentioned earlier, a digital signature is created using public-key cryptography. In public-key cryptography, there are two keys: a private key, known only to its owner, and a public key, which is shared publicly. Unlike encryption—where the sender uses the receiver's public key—in a digital signature the sender uses his own private key to provide authentication, integrity, non-repudiation, and other security properties.

Consider a situation where Alice wants to send a message to Bob (recall the stock characters used in cryptography discussed earlier in this series). Alice generates the digital signature by applying her private key to the message (or more precisely, to the hash of the message). She then sends both the message and the signature to Bob. When Bob receives them, he uses Alice's public key, which is available online, to verify the signature. If the verification is successful, Bob can be certain that the message was indeed sent by Alice, because only Alice has access to her private key.

Note that if Alice wants to encrypt the message as well, she must use Bob's public key, as discussed in a previous article in this series. Additionally, to guarantee message integrity, a cryptographic hash function such as SHA-256, SHA-384, or SHA-512 must be used, since the signature is applied to the hash of the message rather than the message itself.

Now, let us implement a simplified version of a digital signature scheme using RSA and SHA-256. The SageMath program titled 'digsign41.sage' (line numbers included for clarity), shown below, provides a concise yet practically feasible implementation of a digital signature. Note that in this program, the RSA public and private key values are hard-coded, since the detailed process of RSA key generation has already been discussed in earlier articles in this series.

```
1.  import hashlib

2.  n = 95163118457906561534997167608470014334413 57
3.  e = 65537
4.  d = 56178431878449531703084636222302833766986 85

5.  def hash_message(msg):
```